المدرسة الوطنية للعلوم التطبيقية - فاس
ⵜⴰⵙⴷⴰⵡⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵏⴰⵡⵉⵏ ⵜⵉⵜⵜⴰⵡⵉⵏ
École Nationale des Sciences Appliquées de Fès

جامعة سيدي محمد بن عبد الله بفاس
ⵜⴰⵙⴷⴰⵡⵜ ⵙⵉⴷⵉ ⵎⵓⵃⴰⵎⴻⴷ ⴱⵏ ⵄⴱⴷⵍⵍⴰⵀ ⴳ ⴼⴰⵙ
UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH DE FES

PROJET DE FIN D'ANNÉE (PFA)

YO digital SPACE

Refonte Web & Simulateur AI Concierge – Agent Vocal avec Tool Calling et Recherche en Base de Données

Réalisé par :

EL-OGRI ILYASS

Encadré par :

ELMOUDEEN OMAR

PR. SAAD-EDDINE CHAFI

Soutenu le X/10/2026

Année universitaire 2026/2027

Remerciements

L'aboutissement de ce Projet de Fin d'Année marque un jalon fondamental dans la consolidation d'une démarche d'ingénieur. Bien au-delà du cadre d'un exercice académique, cette réalisation m'a permis de confronter l'ensemble des connaissances acquises aux réalités concrètes de l'ingénierie logicielle : de la formalisation des besoins et la conception architecturale jusqu'à l'implémentation, l'optimisation temps réel et la validation fonctionnelle d'un système complexe. À l'issue de ce travail, je tiens à exprimer ma profonde gratitude à toutes celles et ceux qui ont contribué, par leurs conseils, leur expertise et leur soutien, à la réussite de ce projet.

J'adresse mes plus vifs remerciements à mes encadrants pédagogiques pour leur suivi attentif, leur disponibilité constante et la rigueur de leurs orientations. Leurs exigences méthodologiques, leurs remarques constructives et leurs encouragements ont joué un rôle déterminant pour structurer la démarche, clarifier les objectifs et maintenir une progression cohérente face aux différents défis techniques rencontrés.

Je tiens également à exprimer ma reconnaissance à l'ensemble du corps professoral et des intervenants académiques qui ont contribué à ma formation. Les compétences conceptuelles, techniques et humaines transmises tout au long de mon cursus constituent aujourd'hui le socle solide sur lequel s'appuie l'ensemble de ce travail d'ingénierie.

Mes remerciements s'adressent tout particulièrement aux professionnels et collaborateurs qui ont accompagné le projet dans son contexte d'accueil. Les discussions techniques approfondies, les retours d'expérience sur le terrain et la liberté accordée lors des choix de conception ont grandement enrichi ma réflexion, notamment sur les problématiques d'architecture distribuée, de traitement audio à faible latence et d'intégration de modèles d'intelligence artificielle générative.

Enfin, je ne saurais clore ces lignes sans adresser toute ma gratitude à ma famille, à mes proches ainsi qu'à mes collègues. Leur compréhension, leur patience et leurs encouragements indéfectibles tout au long de cette période intense ont été une source constante de motivation et d'énergie, indispensable pour mener à bien ce projet jusqu'à son terme.

Résumé

Le présent rapport décrit la conception et la réalisation d'une plateforme numérique immersive associant une architecture web moderne, un espace de gestion de contenus et un agent conversationnel vocal capable d'interagir en temps réel. Le projet s'inscrit dans une démarche de développement logiciel full-stack et cherche à démontrer comment une expérience numérique peut réunir plusieurs dimensions complémentaires : gestion de contenu, services métier, intelligence artificielle générative, communication bidirectionnelle et rendu graphique avancé.

L'architecture proposée distingue principalement la couche front-end, la couche back-end, la persistance des données et les services externes. Le front-end s'appuie sur React et sur un ensemble d'outils dédiés à la navigation, au rendu graphique, aux animations et à la gestion d'état. Le back-end repose sur NestJS et TypeScript, avec une organisation modulaire, une API REST, un canal WebSocket et un mécanisme de cache en mémoire. La persistance s'appuie sur PostgreSQL via Prisma ORM, tandis que les médias et les notifications sont pris en charge par des services spécialisés.

Une composante essentielle du projet repose sur un agent vocal connecté à une API conversationnelle multimodale. Le flux audio entrant est encodé côté navigateur, transmis par WebSocket, traité par le modèle conversationnel, puis converti en flux audio de sortie. Le système peut parallèlement déclencher des fonctions applicatives pour obtenir les informations nécessaires à la réponse, sans injecter la totalité de la base de connaissances dans le contexte du modèle. Une machine à états émotionnelle permet également d'associer des états d'interaction à la restitution visuelle.

Le rapport présente successivement le contexte et les objectifs, l'analyse des besoins, la conception fonctionnelle et technique, l'environnement de développement, le pipeline d'intelligence artificielle et l'expérience immersive, puis les résultats, la validation, les limites et les perspectives d'évolution.

Mots-clés : architecture logicielle, intelligence artificielle générative, audio temps réel, WebSocket, React, NestJS, PostgreSQL, Prisma, expérience immersive.

Abstract

This report presents the design and implementation of an immersive digital platform combining a modern full-stack architecture, content management capabilities and a real-time conversational voice agent. The project brings together web engineering, real-time communication, generative artificial intelligence and advanced browser graphics within a single technical framework.

The proposed architecture separates the frontend, backend, persistence and external services. React is used for the user interface and interactive experience, while NestJS and TypeScript organize the backend around modular services, REST endpoints and WebSocket communication. PostgreSQL and Prisma provide structured persistence, and dedicated services handle media delivery and email notifications.

A key part of the project is a multimodal conversational pipeline. Incoming audio is captured in the browser, encoded and transmitted through a WebSocket channel. The backend relays the stream to a real-time AI service and handles function calls required to retrieve structured information. The resulting audio stream is sent back to the client, while the frontend synchronizes the conversation with visual states.

The report is structured around six chapters covering the project context, requirements analysis, system design, technology and implementation, real-time AI and immersive interaction, and finally validation, results and future improvements.

Keywords : software architecture, generative AI, real-time audio, WebSocket, React, NestJS, PostgreSQL, Prisma, immersive experience.

Table des matières

Remerciements **2**

Résumé **3**

Abstract **4**

Introduction générale **10**

1 Contexte général et cadrage du projet **12**

I Introduction du chapitre . . . 12
II Contexte du projet . . . 12
III Problématique . . . 13
IV Cahier des charges et objectifs du projet . . . 13
IV.1 Objectif général . . . 13
IV.2 Objectifs fonctionnels . . . 13
IV.3 Objectifs techniques . . . 13
V Enjeux et contraintes . . . 14
VI Périmètre fonctionnel . . . 14
VII Organisation prévisionnelle . . . 14
VIII Conclusion du chapitre . . . 16

2 Analyse des besoins et spécifications **17**

I Introduction du chapitre . . . 17
II Parties prenantes . . . 17
III Besoins fonctionnels . . . 17
III.1 Présentation et contenus . . . 17
III.2 Portfolio et études de cas . . . 18
III.3 Blog . . . 18
III.4 Contact et demandes . . . 18
III.5 Administration de la connaissance . . . 18
III.6 Conversation vocale . . . 18
IV Besoins non fonctionnels . . . 19

V Contraintes particulières de l'audio temps réel . . . 20
VI Référencement naturel et structuration des blocs de contenu . . . 20
VII Spécification des outils conversationnels . . . 21
VIII Priorisation des besoins . . . 22
IX Conclusion du chapitre . . . 22

3 Conception, modélisation UML et architecture du système 23
I Introduction du chapitre . . . 23
II Principes d'architecture . . . 23
III Modélisation UML du système . . . 24
III.1 Diagramme de cas d'utilisation . . . 24
III.2 Modèle UML de la base de données . . . 27
III.3 Dictionnaire synthétique des tables . . . 28
IV Architecture du front-end . . . 28
V Architecture du back-end . . . 29
VI Communication REST . . . 30
VII Communication WebSocket . . . 30
VIII Organisation de la base de données . . . 31
IX Modélisation relationnelle . . . 31
X Flux de mise à jour de la connaissance . . . 31
XI Conclusion du chapitre . . . 32

4 Technologies et réalisation logicielle 33
I Introduction du chapitre . . . 33
II Environnement de développement . . . 33
III Technologies du back-end . . . 33
IV Technologies du front-end . . . 34
V Structure logique des fichiers . . . 35
VI Authentification administrative . . . 36
VII Gestion des médias . . . 36
VIII Qualité du code et maintenabilité . . . 37
IX Conclusion du chapitre . . . 37

5 Intelligence artificielle temps réel et expérience immersive 38
I Introduction du chapitre . . . 38
II Principe du pipeline vocal . . . 38

III Passerelle WebSocket . . . 39
IV Tool Calling . . . 39
V Cache mémoire . . . 39
VI États émotionnels . . . 40
VII Protection contre l'écho et interruption . . . 41
VIII Séquence d'exécution . . . 41
IX Expérience immersive et rendu 3D . . . 41
X Gestion des pages et de l'expérience . . . 42
XI Administration de la base de connaissances . . . 42
XII Conclusion du chapitre . . . 42

6 Réalisation, validation, résultats et perspectives 43
I Introduction du chapitre . . . 43
II Fonctionnalités réalisées . . . 43
III Validation fonctionnelle . . . 44
IV Validation des données . . . 45
V Validation du flux audio . . . 45
VI Résultats obtenus . . . 46
VII Limites identifiées . . . 48
VIII Pistes d'amélioration . . . 49
VIII.1 Limitation des sessions de conversation IA . . . 49
VIII.2 Identification de la session par cookie . . . 49
VIII.3 Vérification complémentaire par adresse IP . . . 49
VIII.4 Contrôle et temporisation côté serveur . . . 49
VIII.5 Réduction de la consommation et maîtrise des coûts . . . 49
VIII.6 Observabilité et suivi de la consommation . . . 50
VIII.7 Tests automatisés . . . 50
IX Bilan technique . . . 50
X Conclusion du chapitre . . . 51

Conclusion générale 52

Webographie 53

Table des figures

1.1 Diagramme de Gantt prévisionnel du projet sur deux mois (8 semaines) . . . 16

3.1 Architecture globale du système . . . 24
3.2 Diagramme UML des cas d'utilisation . . . 25
3.3 Diagramme UML des cas d'utilisation du visiteur . . . 25
3.4 Diagramme UML des cas d'utilisation de l'administrateur . . . 26
3.5 Diagramme UML des cas d'utilisation de l'agent IA vocal . . . 26
3.6 Diagramme UML des entités et de la structure de la base de données . . . 27

4.1 Organisation fonctionnelle du back-end . . . 37

5.1 États émotionnels de l'agent conversationnel . . . 40
5.2 Cycle d'exécution du pipeline audio temps réel et boucle d'orchestration . . . 41

6.1 Résultat de la base de données . . . 46
6.2 Résultat du back-end : structure des modules et services . . . 47
6.3 Résultat du front-end : interfaces et parcours principaux . . . 47
6.4 Résultat de l'espace d'administration . . . 48
6.5 Synthèse des résultats . . . 48

Liste des tableaux

1.1 Périmètre fonctionnel du projet 14
1.2 Découpage méthodologique du projet 15
1.3 Répartition prévisionnelle du travail sur deux mois 15

2.1 Principales parties prenantes 17
2.2 Synthèse des besoins fonctionnels 19
2.3 Besoins non fonctionnels 20
2.4 Rôle des principaux blocs de contenu dans la stratégie SEO 21
2.5 Outils exposés à l'agent conversationnel 21
2.6 Priorisation fonctionnelle 22

3.1 Tables principales de la base de données 28
3.2 Composants principaux du front-end 29
3.3 Modules principaux du back-end 30
3.4 Principales familles d'API identifiées 30
3.5 Entités principales de la persistance 31

4.1 Environnement et outils de développement 33
4.2 Bibliothèques principales du back-end 34
4.3 Bibliothèques principales du front-end 35
4.4 Découpage logique du projet 36

5.1 Principales caractéristiques du flux audio 38
5.2 Fonctions du mécanisme de Tool Calling 39
5.3 Etats visuels associés aux interactions 40
5.4 Domaines administrables dans la base de connaissances 42

6.1 Bilan fonctionnel de la réalisation 44
6.2 Plan de tests fonctionnels 45
6.3 Critères de validation du temps réel 46
6.4 Synthèse des améliorations proposées 50

Introduction générale

La transformation numérique ne se limite plus à la mise en ligne d'informations ou au remplacement d'un support papier par une interface web. Les plateformes contemporaines doivent désormais répondre à des attentes multiples : disponibilité des contenus, personnalisation, fluidité des échanges, accès depuis différents terminaux et capacité à évoluer rapidement. Dans ce contexte, l'ingénierie logicielle tend à rapprocher le développement applicatif, les services cloud, les interfaces immersives et les modèles d'intelligence artificielle.

Le projet étudié dans ce rapport s'inscrit précisément dans cette convergence. Il vise à concevoir une expérience numérique capable de présenter une activité de manière interactive tout en intégrant un agent conversationnel vocal en temps réel. La plateforme doit donc être à la fois un produit web, un système d'information, une interface d'administration et un point d'accès vers des capacités conversationnelles avancées.

Le besoin ne porte pas uniquement sur la création d'une interface visuellement différenciante. Il concerne également la structuration du contenu métier, la séparation claire des responsabilités techniques, l'administration des informations exploitées par l'agent conversationnel, la protection des accès et la gestion d'un canal de communication temps réel. Cette combinaison impose une réflexion globale sur l'architecture.

La démarche adoptée repose sur plusieurs principes. Le premier consiste à séparer les responsabilités : l'interface utilisateur ne doit pas porter seule la logique métier, tandis que le modèle d'intelligence artificielle ne doit pas être utilisé comme une base de données documentaire brute. Le deuxième principe est la modularité : chaque domaine fonctionnel doit pouvoir évoluer sans affecter l'ensemble du système. Le troisième est la réactivité : le temps de réponse doit rester suffisamment faible pour conserver une interaction naturelle, en particulier dans le cas de l'audio.

Le rapport est organisé en six chapitres. Le premier présente le contexte, la problématique, les objectifs et les contraintes. Le deuxième détaille l'analyse des besoins, le périmètre et les exigences. Le troisième présente la conception fonctionnelle et l'architecture du système. Le quatrième traite de l'environnement technologique et de la réalisation logicielle. Le cinquième se concentre sur le pipeline d'intelligence artificielle, l'audio temps réel et l'expérience immersive. Enfin, le sixième présente la validation, les résultats, les limites et les perspectives d'évolution.

Une attention particulière est accordée à la cohérence entre la partie fonctionnelle et la partie technique. Chaque fonctionnalité est associée à des données, à un traitement serveur et, lorsque cela est nécessaire, à une interaction de l'interface. Cette manière de raisonner permet d'éviter une conception centrée uniquement sur l'apparence et de replacer les choix techniques dans leur objectif réel.

Le rapport distingue également la connaissance métier, qui évolue régulièrement, du code applicatif qui fournit les mécanismes de traitement. Cette séparation est particulièrement importante pour l'agent conversationnel : une nouvelle information métier doit pouvoir être ajoutée ou corrigée sans demander une modification du cœur logiciel. Cette exigence influence directement la modélisation de la base de données, le fonctionnement du cache et la conception du module d'administration.

Enfin, l'approche retenue privilégie une progression mesurable. Le fonctionnement d'un module peut être validé indépendamment, puis intégré au parcours complet. Les scénarios de test du dernier chapitre reprennent cette logique en distinguant les opérations classiques, l'administration et le flux vocal temps réel.

Organisation du rapport

La progression retenue reprend une logique classique d'ingénierie : comprendre le besoin, formaliser les exigences, concevoir le système, réaliser les composants, évaluer le résultat puis identifier les pistes d'amélioration. Cette structure facilite la lecture et rend les liens entre les décisions techniques et les objectifs fonctionnels plus explicites.

Chapitre 1 :

Contexte général et cadrage du projet

I Introduction du chapitre

Avant toute phase de développement, un projet logiciel doit être replacé dans son contexte d'usage. Cette première étape permet de comprendre l'origine du besoin, la valeur attendue, les contraintes existantes et les résultats recherchés. Dans le cas présent, le contexte combine une ambition de repositionnement numérique, des besoins de présentation de services et la volonté d'intégrer une expérience conversationnelle plus riche qu'un simple formulaire ou chatbot textuel.

Le cadrage constitue également un moyen de distinguer les éléments essentiels des éléments secondaires. Une plateforme peut proposer de nombreuses fonctionnalités, mais toutes ne possèdent pas le même impact sur la qualité de l'expérience ou sur la complexité globale du système. Une hiérarchisation est donc nécessaire afin de concentrer les efforts sur les composants qui portent réellement la valeur du projet.

II Contexte du projet

Le projet s'inscrit dans une démarche de transformation d'une activité numérique vers un positionnement fortement orienté vers l'ingénierie logicielle, l'automatisation et l'intelligence artificielle. La plateforme doit permettre de présenter les services, les réalisations, les offres et les informations générales dans une expérience moderne et interactive.

Cette évolution implique d'aller au-delà d'une vitrine statique. Les spécifications prévoient une présentation structurée du portfolio, un espace éditorial, des contenus administrables et un agent conversationnel capable de renseigner les visiteurs sur les informations disponibles. Le système doit également maintenir une cohérence entre les informations présentées par l'interface publique et celles exploitées par l'agent intelligent.

L'expérience recherchée est donc à la fois informative et démonstrative. L'utilisateur doit pouvoir découvrir les services et les réalisations, comprendre le positionnement de l'organisation et, lorsque cela est pertinent, engager une conversation vocale. La plateforme devient ainsi une démonstration fonctionnelle des technologies qu'elle cherche à présenter.

III Problématique

La problématique centrale peut être formulée de la manière suivante : comment concevoir une plateforme web immersive et administrable capable de présenter des informations métier structurées tout en assurant une interaction vocale bidirectionnelle en temps réel avec un agent d'intelligence artificielle ?

Cette problématique se décompose en plusieurs questions secondaires. Il faut d'abord déterminer une organisation front-end et back-end capable de supporter des fonctionnalités classiques et un canal WebSocket temps réel. Il faut ensuite structurer les données de manière suffisamment flexible pour représenter les projets, les articles, les services, les offres et les questions fréquentes. Il faut également permettre à l'agent conversationnel de récupérer ces informations à la demande sans dépendre d'un contexte excessivement volumineux.

Enfin, la qualité de l'expérience dépend fortement de la synchronisation entre l'audio, les réponses du modèle, l'état visuel de l'avatar et les interactions de l'utilisateur. La conception doit donc considérer simultanément les aspects logiciels, réseau, données et expérience utilisateur.

IV Cahier des charges et objectifs du projet

IV.1 Objectif général

L'objectif général consiste à développer une plateforme numérique moderne, modulaire et évolutive intégrant une expérience immersive ainsi qu'un agent vocal intelligent fonctionnant en temps réel.

IV.2 Objectifs fonctionnels

Les objectifs fonctionnels identifiés dans les spécifications peuvent être regroupés en six axes :

1. présenter clairement les activités, services et réalisations ;
2. proposer un espace éditorial permettant de publier et consulter des articles ;
3. offrir un mécanisme de contact et de demande d'information ;
4. fournir une interface d'administration pour maintenir les contenus ;
5. intégrer une interaction conversationnelle vocale en temps réel ;
6. maintenir une cohérence entre les données métier et les réponses de l'agent intelligent.

IV.3 Objectifs techniques

Sur le plan technique, le projet cherche à mettre en œuvre une architecture moderne autour de React, NestJS, TypeScript, PostgreSQL, Prisma, WebSocket et des outils de rendu graphique tels que Three.js, OGL et GLSL. Il vise également une gestion propre de l'état côté client, une validation des données côté serveur et une séparation nette entre les modules métier.

V Enjeux et contraintes

Le principal enjeu est de concilier richesse fonctionnelle et faible latence. Un système capable de gérer une interface complexe mais lent dans ses interactions temps réel ne répondrait pas au besoin d'une conversation naturelle. La conception doit donc réduire autant que possible les opérations inutiles, notamment les accès répétés à la base de données pendant les échanges vocaux.

Un autre enjeu est la maintenabilité. Les données utilisées par l'agent évoluent avec l'activité : les services changent, les prix peuvent être révisés, le portfolio s'enrichit et les questions fréquentes sont susceptibles d'être modifiées. L'architecture doit donc éviter de figer ces éléments dans le code source.

La sécurité représente également une contrainte importante. L'administration nécessite un mécanisme d'accès protégé, tandis que les contenus et informations sensibles doivent être séparés des composants exposés au public. La gestion des sessions, des codes temporaires, des accès API et des fichiers doit être pensée dès la phase de conception.

VI Périmètre fonctionnel

Table 1.1 Périmètre fonctionnel du projet

Domaine	Fonctionnalités principales	Finalité
Présentation	Accueil, services, réalisations, informations générales	Informer et orienter le visiteur
Contenu éditorial	Blog, catégories, détail des articles	Valoriser les contenus et la connaissance métier
Relation	Contact, demande de devis, collecte des besoins	Faciliter la prise de contact
Administration	Projets, articles, contacts, base de connaissances	Maintenir les contenus sans modification du code
Conversation	Voix, WebSocket, appels d'outils, états émotionnels	Permettre une interaction naturelle en temps réel
Expérience	Animations, 3D, transitions, scroll fluide	Renforcer l'immersion et la différenciation

VII Organisation prévisionnelle

Le fichier de spécification indique une réalisation sur une période courte, ce qui implique une organisation structurée des activités. Une organisation logique peut être décrite selon les étapes suivantes : analyse, conception, modélisation des données, mise en place du back-end,

développement du front-end, intégration des services externes, tests et préparation de la démonstration.

Table 1.2 Découpage méthodologique du projet

N°	Phase	Livrables attendus
1	Analyse et cadrage	Objectifs, périmètre, exigences, risques
2	Conception	Architecture, modèle de données, interfaces logiques
3	Infrastructure de données	Schéma relationnel, services d'accès, données initiales
4	Back-end	Modules, API REST, WebSocket, sécurité
5	Front-end	Pages, composants, navigation, administration
6	IA temps réel	Session multimodale, outils, audio, interruptions
7	Validation	Tests fonctionnels, tests de flux, ajustements
8	Finalisation	Documentation, préparation de démonstration et bilan

Table 1.3 Répartition prévisionnelle du travail sur deux mois

Phase	**Activité**	S1	S2	S3	S4	S5	S6	S7	S8
1	**Analyse et cadrage**	•	•						
2	**Conception UML et base de données**		•	•					
3	**Mise en place du back-end**			•	•	•			
4	**Développement du front-end**				•	•	•	•	
5	**IA, WebSocket et intégration**					•	•	•	
6	**Tests, corrections et validation**							•	•
7	**Documentation**								•

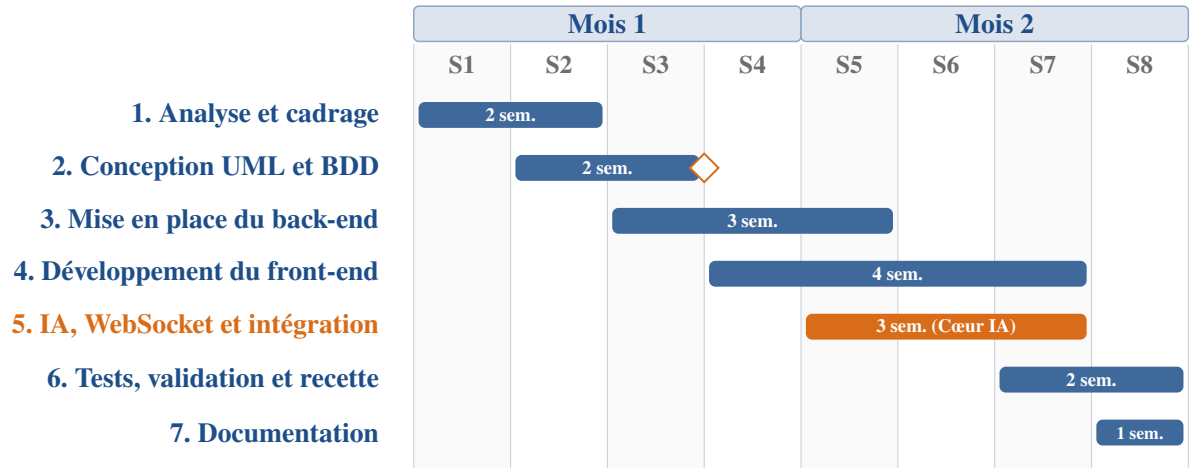


Figure 1.1 Diagramme de Gantt prévisionnel du projet sur deux mois (8 semaines)

VIII Conclusion du chapitre

L'étude du contexte montre que le projet ne peut pas être réduit à la construction d'une simple interface web. Il s'agit d'un système transversal combinant présentation de contenu, administration, persistance, communication en temps réel et intelligence artificielle. Cette diversité justifie une architecture modulaire et prépare naturellement la phase d'analyse détaillée des besoins.

Chapitre 2 :

Analyse des besoins et spécifications

I Introduction du chapitre

L'analyse des besoins transforme une vision globale en exigences concrètes. Elle permet de préciser ce que le système doit faire, pour qui, avec quelles contraintes et dans quelles limites. Cette étape est essentielle dans un projet qui combine plusieurs domaines techniques, car elle évite de confondre les fonctionnalités visibles avec les mécanismes nécessaires à leur fonctionnement.

II Parties prenantes

Le système met en jeu plusieurs catégories d'acteurs. Les visiteurs consultent les contenus publics et peuvent utiliser l'agent conversationnel. Les administrateurs disposent d'un espace sécurisé permettant de modifier les informations exploitées par la plateforme. Les services externes représentent enfin des acteurs techniques indirects, puisqu'ils fournissent la messagerie, le stockage ou le traitement conversationnel.

Table 2.1 Principales parties prenantes

Acteur	Besoin principal	Interaction avec le système
Visiteur	Découvrir les services et obtenir des informations	Navigation, lecture, contact, conversation vocale
Administrateur	Maintenir les contenus et la connaissance métier	Authentification, CRUD, validation, publication
Équipe technique	Faire évoluer la plateforme	Code, données, tests, maintenance
Services externes	Exécuter une fonction spécialisée	API, SMTP, stockage média, IA conversationnelle

III Besoins fonctionnels

III.1 Présentation et contenus

La plateforme doit présenter les informations de manière structurée. Les services peuvent être organisés par catégorie, les projets disposent d'un contenu détaillé et les articles peuvent être

consultés séparément. La structure des données prévoit également un profil général permettant de centraliser la présentation, la philosophie, la mission et la vision.

III.2 Portfolio et études de cas

Le domaine des projets constitue une composante importante de la plateforme. Chaque projet possède un identifiant stable, un slug, des médias associés et des informations textuelles en plusieurs langues. Les informations de contexte, de solution, de fonctionnalités et de résultats sont conservées dans des structures JSON afin de garder une certaine souplesse.

III.3 Blog

Le module éditorial doit permettre de créer, modifier et publier des articles. Les métadonnées comprennent notamment la date, la catégorie, le temps de lecture estimé et l'image de couverture. Le contenu rédactionnel est stocké selon une structure compatible avec plusieurs langues.

III.4 Contact et demandes

Le formulaire de contact reçoit le nom, l'organisation éventuelle, l'adresse électronique, le téléphone, le besoin exprimé et le message. Un statut permet de distinguer les demandes non lues, lues et traitées. Une notification électronique peut être déclenchée pour alerter l'équipe lorsqu'un nouveau contact est enregistré.

III.5 Administration de la connaissance

La base de connaissances administrable constitue l'un des besoins les plus importants du projet. L'administrateur doit pouvoir modifier les services, les tarifs, le profil, la FAQ et les autres informations susceptibles d'être utilisées pendant une conversation. La mise à jour doit être répercutée rapidement dans le cache utilisé par le service conversationnel.

III.6 Conversation vocale

L'utilisateur doit pouvoir ouvrir une interaction vocale sans passer par un formulaire textuel traditionnel. Le navigateur capture l'audio, l'encode selon le format attendu par le service distant et transmet les données au back-end. Les réponses audio sont ensuite restituées dans le navigateur.

Table 2.2 Synthèse des besoins fonctionnels

ID	Besoin	Description
BF-01	Consulter les contenus	Accéder aux services, réalisations et informations générales
BF-02	Consulter le blog	Parcourir les articles et accéder au détail d'une publication
BF-03	Envoyer une demande	Saisir et transmettre un besoin via un formulaire
BF-04	Administrer les projets	Créer, modifier et supprimer les contenus du portfolio
BF-05	Administrer les articles	Gérer les contenus éditoriaux et leurs métadonnées
BF-06	Administrer la connaissance	Modifier services, prix, FAQ, profil et informations métier
BF-07	Interagir vocalement	Échanger avec l'agent via audio temps réel
BF-08	Exploiter les outils	Autoriser le modèle à obtenir les informations nécessaires à la réponse
BF-09	Synchroniser l'état visuel	Refléter l'état émotionnel de l'agent côté client
BF-10	Protéger l'administration	Contrôler l'accès par une authentification adaptée

IV Besoins non fonctionnels

Les besoins non fonctionnels décrivent la qualité attendue du système. Dans le contexte d'une interaction audio, la latence constitue une contrainte particulièrement importante. Le système doit également rester maintenable, modulaire, responsive et suffisamment sécurisé pour limiter les accès non autorisés.

Table 2.3 Besoins non fonctionnels

Critère	Attente	Conséquence architecturale
Performance	Réduire les délais de réponse	Cache RAM, flux WebSocket, traitement asynchrone
Disponibilité	Assurer un fonctionnement stable	Services découplés, gestion des erreurs
Sécurité	Protéger les accès d'administration	OTP, validation, séparation des responsabilités
Évolutivité	Ajouter de nouveaux contenus et services	Architecture modulaire, schéma de données extensible
Maintenabilité	Faciliter les évolutions	TypeScript, modules NestJS, stores dédiés
Responsive	Adapter l'interface aux usages	Composants réutilisables et règles responsive
Expérience	Préserver la fluidité	Optimisation du rendu et contrôle des animations

V Contraintes particulières de l'audio temps réel

Le traitement vocal introduit une contrainte différente des requêtes HTTP classiques. Une page peut tolérer un petit délai lors du chargement d'une ressource, tandis qu'une conversation devient rapidement artificielle si chaque tour de parole est interrompu par une longue attente. Le système doit donc réduire le nombre de traitements intermédiaires et limiter les accès coûteux.

La spécification retient l'utilisation d'un cache mémoire préchargé pour les données de la base de connaissances. Cette stratégie vise à réduire le nombre de requêtes pendant les échanges et à répondre rapidement aux appels d'outils. Le cache ne remplace pas la base persistante : il joue le rôle d'une copie de consultation rapide.

VI Référencement naturel et structuration des blocs de contenu

Le cahier des charges doit également intégrer une dimension de référencement naturel (SEO). Les blocs de contenu du site ne sont donc pas uniquement pensés comme des éléments visuels : ils doivent contribuer à la compréhension du contenu par les visiteurs et par les moteurs de recherche. La structure éditoriale doit favoriser des titres hiérarchisés, des contenus descriptifs, des URL cohérentes, des métadonnées pertinentes, des liens internes et une organisation sémantique régulière.

Cette exigence concerne notamment les blocs de présentation, les services, les études de

cas, les articles du blog et les questions fréquentes. Chaque bloc devra porter une information utile et identifiable, plutôt que de constituer uniquement un élément décoratif. L'espace éditorial doit ainsi pouvoir soutenir une stratégie de publication régulière et améliorer la visibilité organique du site.

Table 2.4 Rôle des principaux blocs de contenu dans la stratégie SEO

Bloc	Fonction éditoriale	Contribution au référencement
Présentation	Positionnement et proposition de valeur	Clarification du sujet principal et des termes métier
Services	Description structurée des offres	Ciblage de requêtes thématiques et intentionnelles
Portfolio	Études de cas et réalisations	Enrichissement sémantique et preuve par le contenu
Blog	Publications techniques et métier	Création régulière de contenus indexables
FAQ	Réponses aux questions fréquentes	Couverture des intentions de recherche et longue traîne
Contact	Informations pratiques et conversion	Cohérence des informations locales et commerciales

VII Spécification des outils conversationnels

Le modèle peut déclencher un ensemble de fonctions dédiées. Chaque fonction correspond à une catégorie d'information et évite de surcharger le contexte global.

Table 2.5 Outils exposés à l'agent conversationnel

`Fonction`	Rôle
`getCompanyInfo()`	Retourner le profil, la mission, la vision et les informations générales
`getServices()`	Fournir la liste des services et leur description
`getPricing()`	Retourner les forfaits, tarifs indicatifs et délais disponibles
`getProjects()`	Accéder aux informations du portfolio et aux études de cas
`getFaq()`	Répondre à partir des questions fréquentes structurées
`setEmotion()`	Définir l'état émotionnel destiné à être reflété sur le front-end

VIII Priorisation des besoins

Une priorisation peut être utilisée afin de distinguer le socle nécessaire au fonctionnement du produit des fonctionnalités qui enrichissent l'expérience. Le cœur fonctionnel comprend la présentation, l'administration des contenus, la persistance et la conversation. Les effets graphiques avancés, les optimisations supplémentaires et certaines fonctions de confort peuvent être considérés comme des éléments de renforcement.

Table 2.6 Priorisation fonctionnelle

Priorité	**Domaine**	Justification
P1	**Données et persistance**	Support du contenu et des informations métier
P1	**API et logique métier**	Coordination des fonctionnalités
P1	**Conversation temps réel**	Fonction distinctive du projet
P1	**Administration**	Mise à jour des données sans modifier le code
P2	**Expérience 3D et animations**	Valorisation du produit et différenciation
P2	**Automatisation éditoriale**	Amélioration de l'exploitation des contenus
P3	**Optimisations de production**	Amélioration de la montée en charge et du mobile

IX Conclusion du chapitre

L'analyse met en évidence un système à plusieurs dimensions. Les exigences fonctionnelles définissent ce que les utilisateurs doivent pouvoir faire, tandis que les contraintes non fonctionnelles déterminent la manière dont le système doit se comporter. La priorité accordée à la réactivité, à la modularité et à la maintenabilité justifie les choix architecturaux présentés dans le chapitre suivant.

Chapitre 3 :

Conception, modélisation UML et architecture du système

I Introduction du chapitre

La conception transforme les exigences en une organisation technique cohérente. L'enjeu principal est d'établir des frontières claires entre les différents composants. La plateforme décrite dans les spécifications repose sur plusieurs sous-systèmes qui communiquent par des interfaces explicites : interface web, API métier, persistance, services externes et canal temps réel.

II Principes d'architecture

L'architecture générale suit une séparation en couches. La couche de présentation regroupe l'interface React, la navigation, les composants visuels et les mécanismes d'interaction. La couche serveur contient les modules NestJS, les contrôleurs REST, les services métier et la passerelle WebSocket. La couche de données fournit la persistance via PostgreSQL et Prisma. Enfin, plusieurs services spécialisés prennent en charge les médias, les courriels et le traitement conversationnel.

Cette séparation permet de limiter le couplage entre les composants. Le front-end ne communique pas directement avec la base de données. Le modèle de langage n'a pas besoin d'accéder directement à la base : il passe par des outils définis côté serveur. L'administration manipule les mêmes données que les fonctionnalités publiques, mais dans un espace sécurisé.

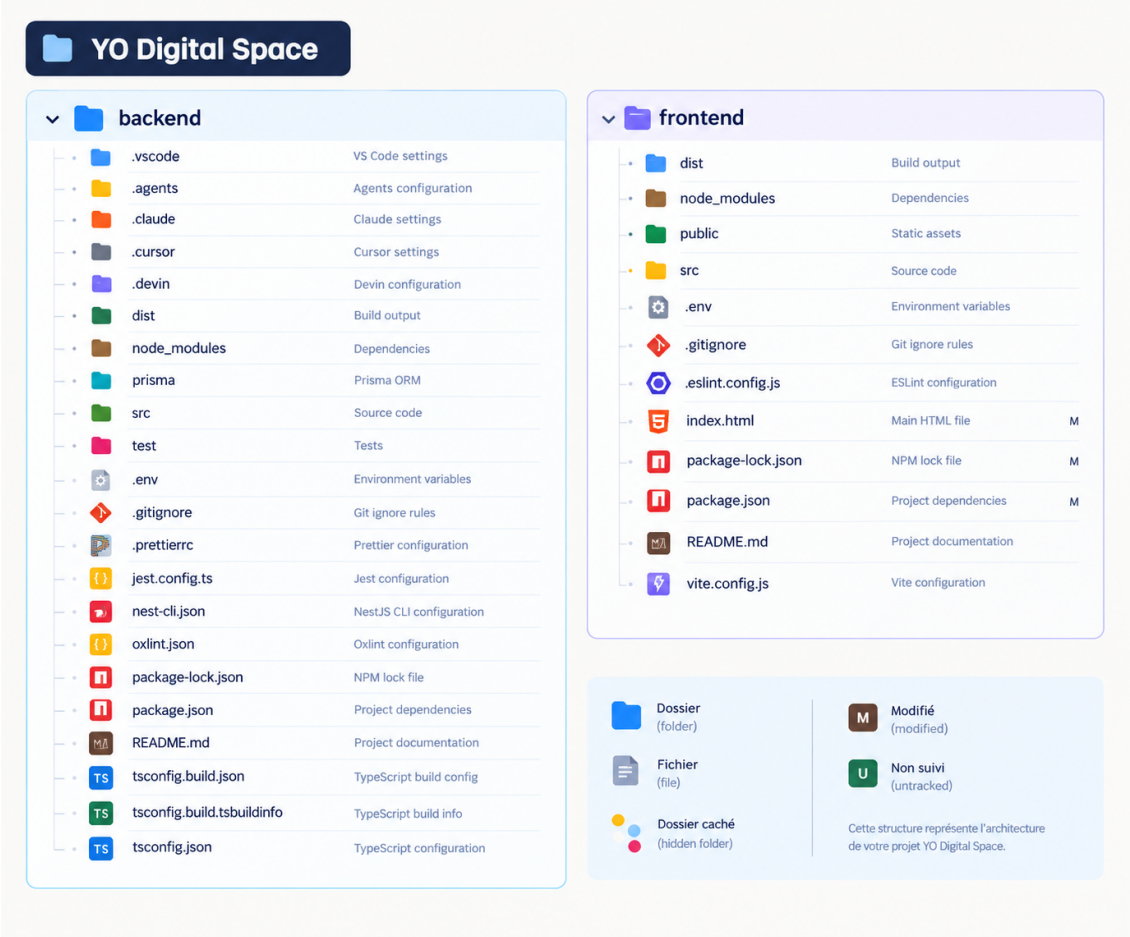


Figure 3.1 Architecture globale du système

III Modélisation UML du système

La conception fonctionnelle doit être accompagnée d'une modélisation UML permettant de représenter les acteurs, les fonctionnalités principales et les relations entre les composants. Cette étape servira de référence pour la réalisation et pour la lecture du système par un lecteur non spécialiste du code.

III.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présentera les principaux acteurs du système, notamment l'utilisateur public et l'administrateur, ainsi que leurs interactions avec les fonctions de consultation, de contact, de gestion des contenus, d'authentification et de maintenance de la base de connaissances.

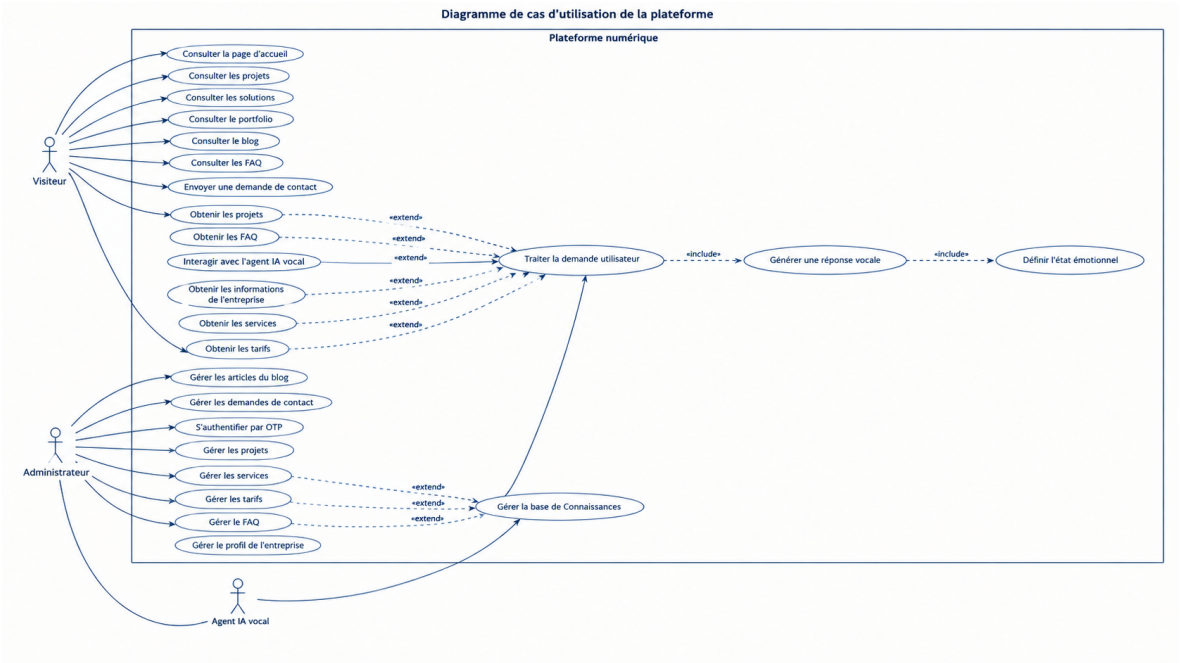


FIGURE 3.2 Diagramme UML des cas d'utilisation

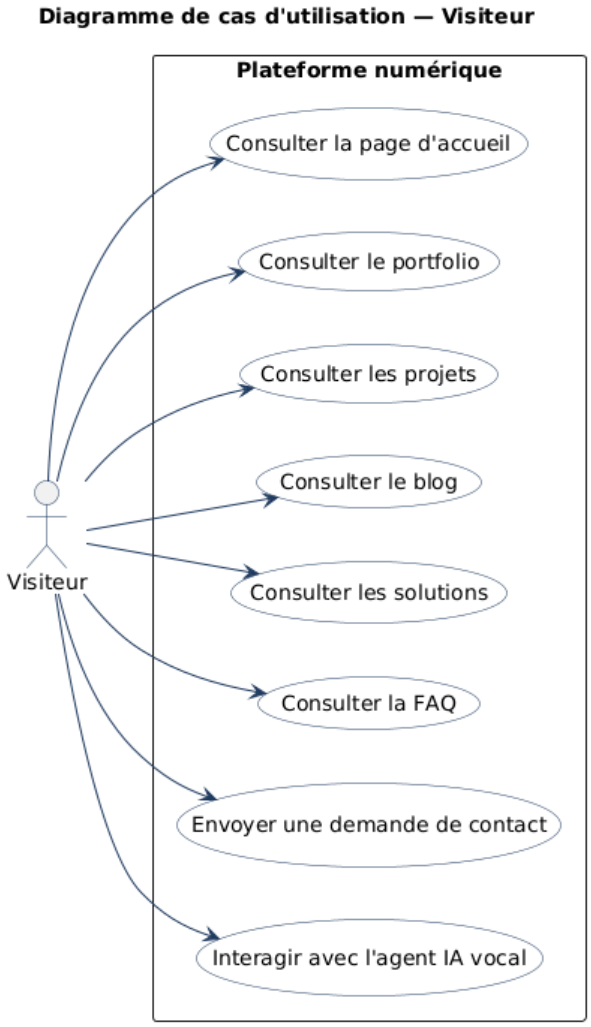


FIGURE 3.3 Diagramme UML des cas d'utilisation du visiteur

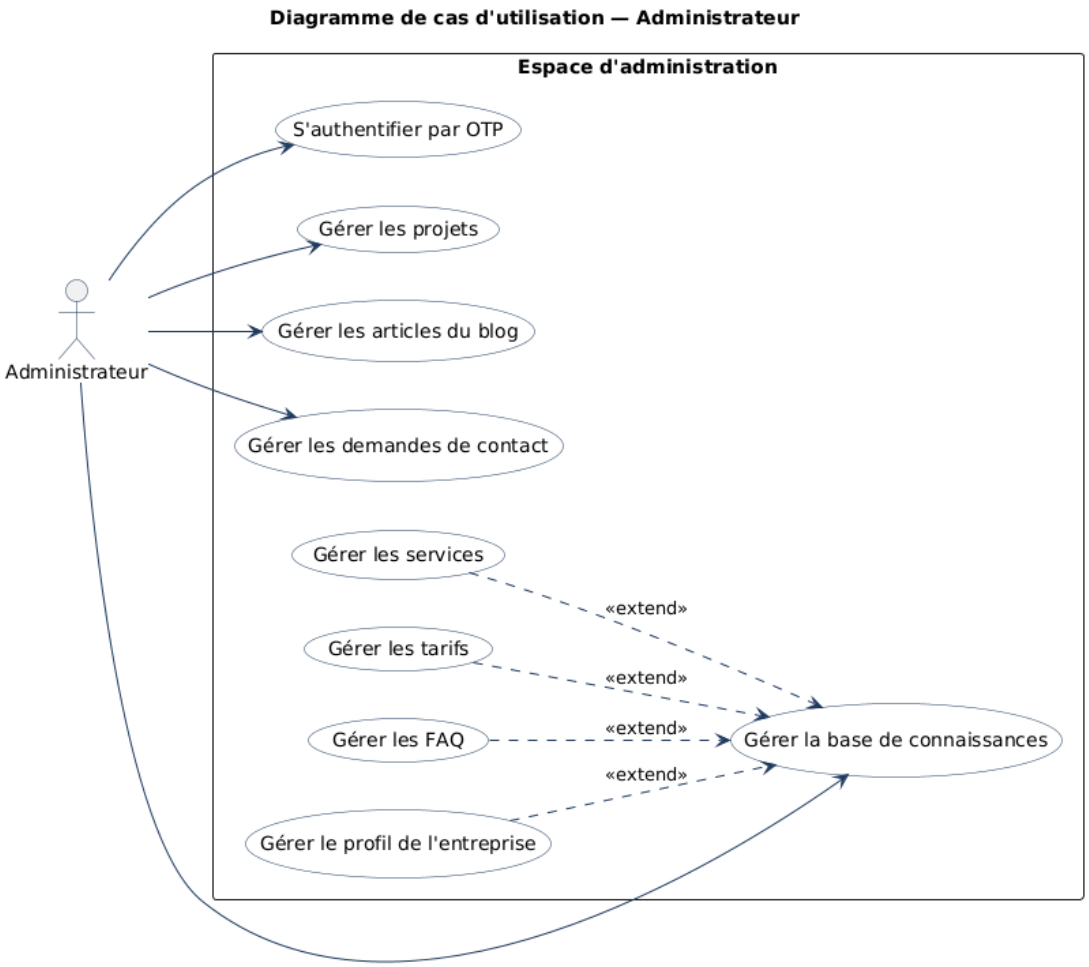


FIGURE 3.4 Diagramme UML des cas d'utilisation de l'administrateur

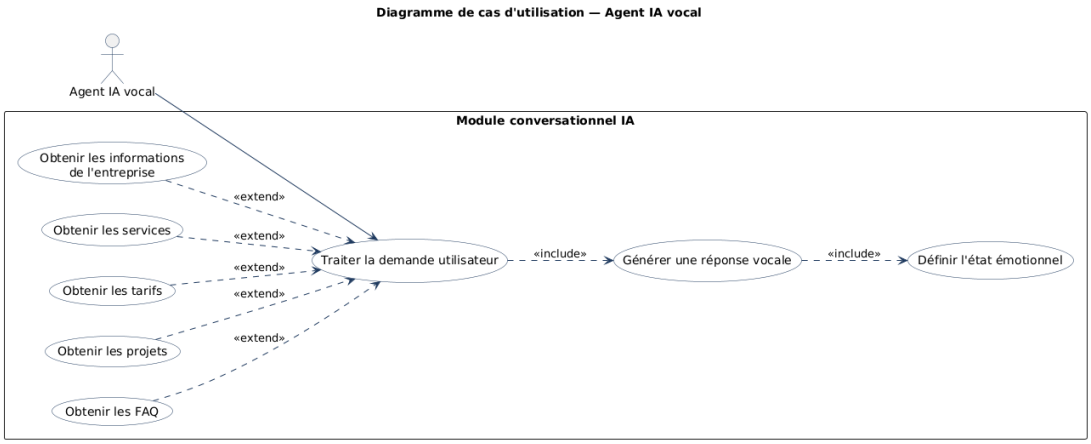


FIGURE 3.5 Diagramme UML des cas d'utilisation de l'agent IA vocal

III.2 Modèle UML de la base de données

La conception de la base de données sera présentée sous une forme UML afin de rendre visibles les principales classes métier, leurs attributs, leurs identifiants et les relations entre elles. Cette représentation complétera le modèle physique issu de Prisma et facilitera la lecture des dépendances entre les différentes tables.

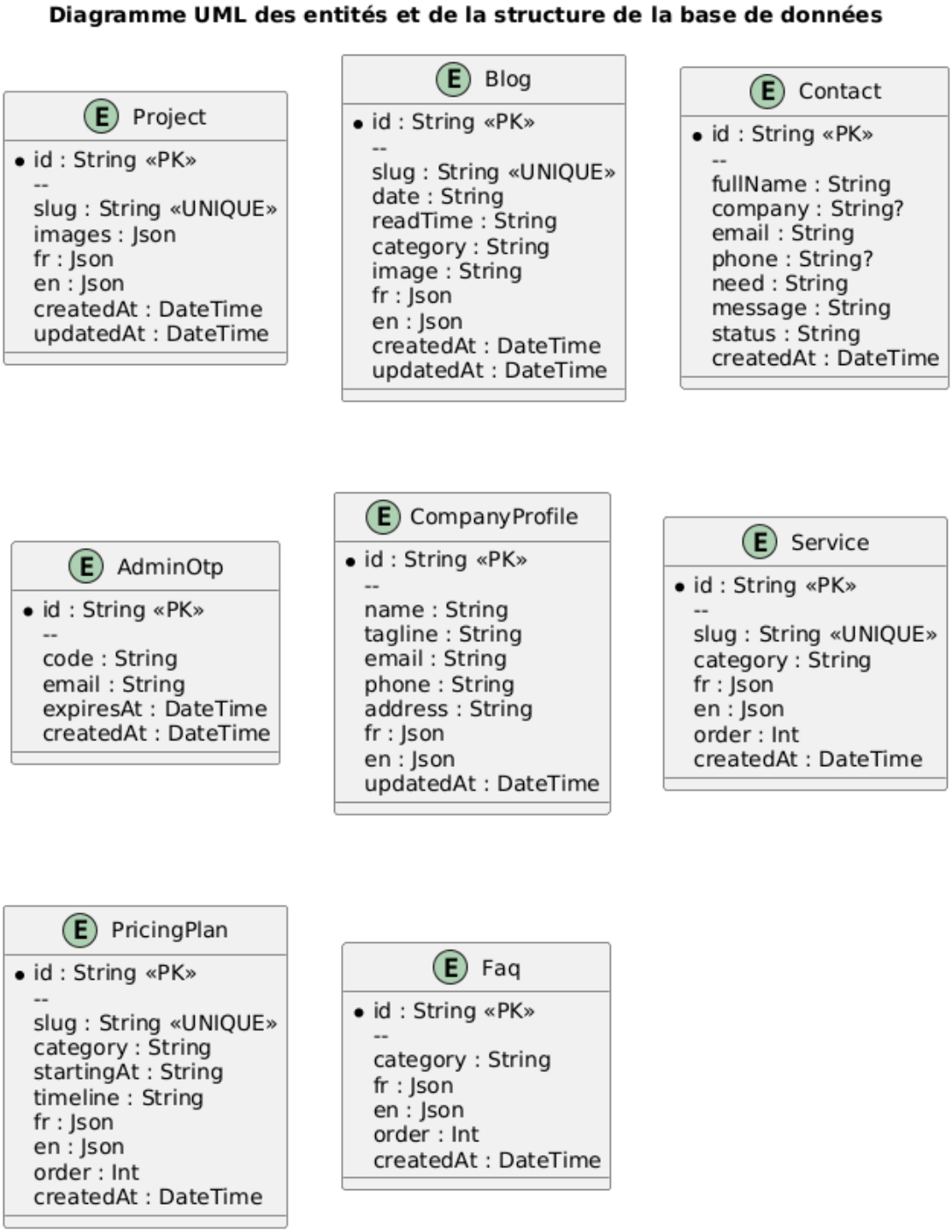


Figure 3.6 Diagramme UML des entités et de la structure de la base de données

III.3 Dictionnaire synthétique des tables

Le modèle relationnel comprend plusieurs entités complémentaires. Le tableau suivant constitue la base documentaire qui accompagnera ultérieurement le diagramme UML détaillé.

Table 3.1 Tables principales de la base de données

Table	Finalité	Données principales
Project	Gestion du portfolio	Identifiant, slug, médias, contenus, dates
Blog	Gestion éditoriale	Titre, extrait, contenu, catégorie, date, langue
Contact	Demandes et devis	Nom, organisation, email, téléphone, besoin, message, statut
AdminOtp	Authentification temporaire	Code, email, expiration, date de création
CompanyProfile	Profil et positionnement	Nom, coordonnées, mission, vision, philosophie
Service	Catalogue des services	Slug, catégorie, titre, description, fonctionnalités
PricingPlan	Offre et tarification	Catégorie, tarif indicatif, délai, livrables
Faq	Questions fréquentes	Catégorie, question, réponse, ordre

IV Architecture du front-end

Le front-end est développé avec React. La structure est organisée autour de pages, composants, hooks, utilitaires, routes et stores. Le routeur gère la navigation de l'application et l'espace d'administration. Zustand est utilisé pour maintenir certains états globaux tels que l'authentification, les projets, les articles, les contacts et les informations de connaissance.

La partie immersive s'appuie sur plusieurs technologies complémentaires. Three.js et React Three Fiber assurent le rendu 3D déclaratif, tandis que GLSL permet de personnaliser les shaders. OGL est utilisé pour le carrousel graphique cylindrique, et GSAP intervient dans les animations et les effets liés au défilement. Lenis assure une sensation de scroll fluide et contribue à la cohérence visuelle de l'expérience.

Table 3.2 Composants principaux du front-end

Composant / zone	Responsabilité
App.jsx	Routage général, structure de l'application, comportements globaux
LandingPage	Assemblage des sections principales de la page d'accueil
Portfolio	Liste et détail des études de cas
Blog	Consultation et lecture des articles
Solution	Présentation des solutions métier
Contact	Saisie et transmission des demandes
CeoSimulator	Interface immersive de conversation vocale
Admin	Gestion des contenus, contacts et connaissances
Stores Zustand	Gestion d'état partagée
Hooks / utils	Communication audio et logique réutilisable

V Architecture du back-end

Le back-end repose sur NestJS et TypeScript. L'approche modulaire facilite la séparation entre les domaines fonctionnels. Les modules identifiés dans les spécifications comprennent notamment la persistance, les projets, le blog, le courrier, les contacts, le stockage média, l'authentification, la base de connaissances et le module conversationnel.

Le point d'entrée configure l'application, le contrôle des données et le support WebSocket. Les contrôleurs exposent les routes REST tandis que les services contiennent les traitements métier. Cette organisation réduit la logique dans les contrôleurs et rend chaque domaine plus simple à maintenir.

Table 3.3 Modules principaux du back-end

Module	Responsabilité
PrismaModule	Accès à PostgreSQL via Prisma
ProjectsModule	Gestion du portfolio
BlogModule	Gestion des articles
MailModule	Envoi des courriels
ContactModule	Réception des demandes
CloudinaryModule	Gestion des médias distants
AuthModule	Authentification administrative
KnowledgeModule	Services, tarifs, FAQ et profil
CeoModule	WebSocket, conversation et appels d'outils

VI Communication REST

Les API REST servent à gérer les opérations classiques de consultation et de modification. Les projets, les articles, les contacts et les informations de connaissance peuvent ainsi être exposés via des routes dédiées. Les réponses sont structurées sous forme JSON afin d'être facilement exploitées par le client React.

Table 3.4 Principales familles d'API identifiées

Domaine	Opérations	Données principales
/projects	lecture et CRUD	portfolio, médias, contenu multilingue
/blog	lecture et CRUD	articles, catégories, métadonnées
/knowledge	lecture et CRUD	profil, services, tarifs, FAQ
/contact	création et suivi	messages, besoins, statut
/auth	authentification	codes temporaires et session

VII Communication WebSocket

La communication WebSocket intervient dans le scénario vocal. Contrairement à une succession de requêtes indépendantes, le canal reste ouvert pendant la conversation et permet d'échanger des messages dans les deux sens. Le client envoie des paquets audio au serveur, tandis que le serveur transmet le flux vers le service conversationnel puis retourne les paquets audio de réponse au navigateur.

Ce mécanisme est également utilisé pour transmettre des événements d'état, notamment les changements d'émotion ou les interruptions. La présence d'un canal persistant évite de reconstruire un nouveau contexte réseau pour chaque fragment audio.

VIII Organisation de la base de données

Le modèle Prisma décrit huit grandes familles d'entités dans la spécification : projets, articles, contacts, codes temporaires, profil, services, tarifs et FAQ. Cette organisation couvre à la fois les contenus publics et la connaissance métier utilisée par le système conversationnel.

Table 3.5 Entités principales de la persistance

Entité	Rôle
Project	Informations et médias des études de cas
Blog	Contenus éditoriaux
Contact	Demandes et messages reçus
AdminOtp	Codes temporaires d'accès administratif
CompanyProfile	Profil, philosophie, mission et vision
Service	Offres et catégories de services
PricingPlan	Forfaits, tarifs indicatifs et délais
Faq	Questions fréquentes et réponses structurées

IX Modélisation relationnelle

Les entités possèdent un identifiant unique, des champs structurés et, lorsque cela est nécessaire, des champs JSON permettant de gérer des contenus plus riches. Le choix de conserver certaines informations éditoriales dans des colonnes JSON permet d'éviter une explosion du nombre de tables de traduction tout en gardant une organisation suffisamment claire.

La présence d'un identifiant unique et d'un slug pour plusieurs ressources permet de distinguer les références techniques des références lisibles utilisées dans les URL. Les champs de date facilitent également le tri, l'audit et la gestion des mises à jour.

X Flux de mise à jour de la connaissance

La base de connaissances suit un flux simple : l'administrateur modifie une donnée, le back-end valide la requête, la donnée est persistée puis le cache mémoire est actualisé. Cette architecture permet de séparer la vérité persistante du mécanisme de lecture rapide utilisé par l'agent.

XI Conclusion du chapitre

La conception montre une architecture découpée en responsabilités cohérentes. Les API REST prennent en charge les interactions classiques, WebSocket répond au besoin temps réel et Prisma fournit une couche typée pour la persistance. Cette base architecturale crée un cadre stable pour présenter les technologies et les détails d'implémentation.

Chapitre 4 :

Technologies et réalisation logicielle

I Introduction du chapitre

Le choix des technologies doit répondre à trois objectifs : disposer d'outils suffisamment puissants pour implémenter le système, conserver une architecture lisible et permettre une évolution progressive. Le projet combine donc des technologies de développement généralistes avec des bibliothèques spécialisées dans le rendu graphique et l'audio temps réel.

II Environnement de développement

Le développement s'appuie principalement sur un environnement JavaScript/TypeScript. Le back-end utilise Node.js et NestJS, tandis que le front-end utilise React avec Vite. L'organisation du code côté serveur suit un découpage par modules et services, et le front-end s'appuie sur des composants et stores réutilisables.

Table 4.1 Environnement et outils de développement

Outil / environnement	Utilisation
Node.js	Exécution JavaScript côté serveur
Vite	Build et développement du front-end
VS Code	Édition et organisation du code
PostgreSQL	Persistance relationnelle
Prisma	Accès typé à la base de données
Postman	Vérification des API
Docker	Conteneurisation des services de l'application
Nginx	Reverse proxy et service des fichiers statiques

III Technologies du back-end

NestJS fournit le cadre principal du serveur. Son approche par injection de dépendances favorise la séparation entre contrôleurs, services et fournisseurs. L'adaptateur WebSocket est utilisé pour gérer le flux temps réel. Prisma assure la communication avec PostgreSQL et fournit des types correspondant au schéma de données.

L'authentification administrative repose sur un mécanisme OTP temporaire envoyé par courrier électronique. Le code possède une date d'expiration et doit être consommé une fois la validation effectuée. Le stockage média est délégué à un service spécialisé afin de ne pas faire reposer les fichiers lourds sur le serveur applicatif.

Table 4.2 Bibliothèques principales du back-end

Bibliothèque	Rôle technique
@nestjs/core / @nestjs/common	Cœur du framework NestJS et organisation de l'application.
@nestjs/platform-express	Couche HTTP basée sur Express.
@nestjs/websockets	Intégration WebSocket dans l'application.
@nestjs/platform-ws	Adaptateur WebSocket natif.
ws	Implémentation WebSocket légère.
@prisma/client	Client TypeScript généré pour la base de données.
prisma	Définition et génération du modèle relationnel.
class-validator	Validation des DTO.
class-transformer	Transformation et filtrage des données.
nodemailer	Envoi des courriels transactionnels.
reflect-metadata	Support des décorateurs NestJS.
rxjs	Gestion réactive utilisée par l'écosystème NestJS.
tsx	Exécution de scripts TypeScript.
oxlint	Contrôle statique du code.

IV Technologies du front-end

React constitue la base de l'interface. React Router DOM prend en charge la navigation, Axios assure les requêtes HTTP et Zustand gère l'état partagé. Tailwind CSS facilite l'organisation du style, tandis que Lucide React fournit des icônes cohérentes.

Les fonctionnalités graphiques avancées s'appuient sur Three.js, React Three Fiber, Drei, postprocessing, GLSL et OGL. GSAP et ScrollTrigger pilotent les animations complexes, Lenis ajoute le défilement fluide et Lottie permet d'intégrer certaines animations vectorielles.

Table 4.3 Bibliothèques principales du front-end

Bibliothèque	Rôle technique
react / react-dom	Construction de l'interface utilisateur.
vite	Build et développement rapide.
tailwindcss	Mise en forme utilitaire et responsive.
three	Rendu 3D WebGL.
@react-three/fiber	Intégration déclarative de Three.js dans React.
@react-three/drei	Utilitaires de caméra et de contrôle.
@react-three/postprocessing	Effets graphiques supplémentaires.
vite-plugin-glsl	Chargement des shaders GLSL.
ogl	Rendu WebGL spécifique du carrousel.
gsap / ScrollTrigger	Animations et scénarios liés au défilement.
@studio-freight/lenis	Défilement fluide.
lottie-web	Animations vectorielles.
zustand	Gestion d'état global.
axios	Communication HTTP.
react-router-dom	Routage SPA.
lucide-react	Icônes SVG.
canvas-confetti	Effets de particules sur certaines interactions.

V Structure logique des fichiers

L'organisation des fichiers traduit l'architecture générale. Côté serveur, les dossiers sont regroupés par domaine fonctionnel. Côté client, les pages, composants, hooks et stores sont séparés. Cette structure permet à un développeur de localiser rapidement la partie du code responsable d'une fonctionnalité.

Table 4.4 Découpage logique du projet

Zone	Contenu principal
`backend/src/main.ts`	Entrée et configuration générale
`backend/src/app.module`	Module racine
`backend/src/ceo`	Passerelle et services conversationnels
`backend/src/knowledge`	Gestion des connaissances
`backend/src/projects`	Gestion du portfolio
`backend/src/blog`	Gestion du blog
`backend/src/contact`	Réception des demandes
`backend/src/auth`	Authentification administrative
`Frontend/src/pages`	Pages et écrans principaux
`Frontend/src/components`	Composants réutilisables
`Frontend/src/hooks`	Logique spécialisée, notamment audio
`Frontend/src/store`	Stores Zustand
`prisma/schema.prisma`	Modèle relationnel
`prisma/seed.ts`	Données initiales

VI Authentification administrative

L'authentification administrative ne repose pas sur un mot de passe statique décrit comme permanent. Le système prévoit un code à usage temporaire transmis par courrier électronique. Le code est généré de manière aléatoire, associé à une expiration et supprimé après validation.

Cette approche réduit la surface d'attaque liée aux identifiants persistants. Elle exige toutefois une gestion rigoureuse de l'expiration, des tentatives, des sessions et de la disponibilité du service de messagerie.

VII Gestion des médias

Le stockage des images est externalisé. Lorsqu'un média est reçu par le serveur, le flux est envoyé au service de stockage afin de produire une URL exploitable par le front-end. Cette stratégie limite l'utilisation du disque local et facilite la diffusion des ressources statiques.

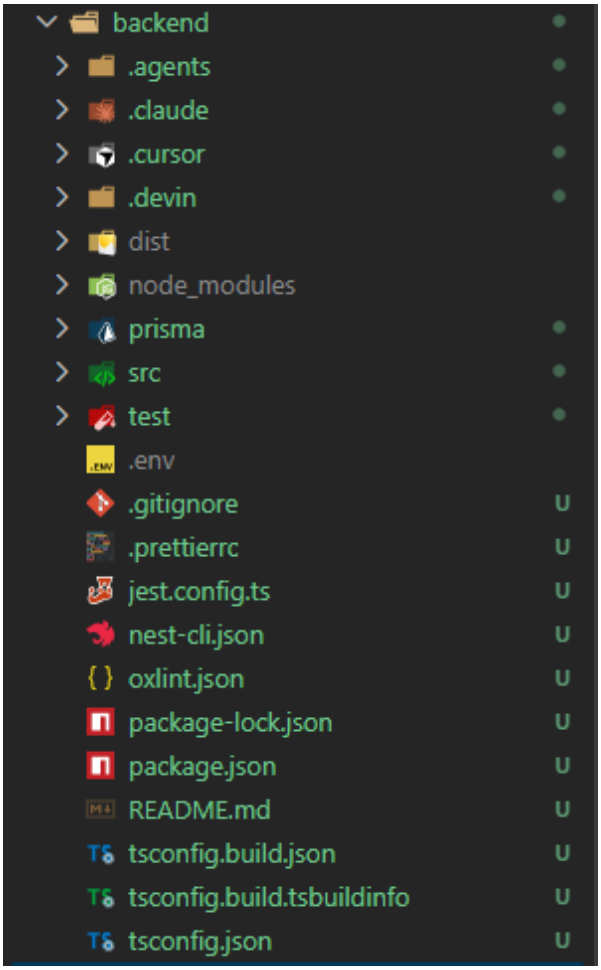


Figure 4.1 Organisation fonctionnelle du back-end

VIII Qualité du code et maintenabilité

Plusieurs choix favorisent la maintenabilité : séparation par domaines, types TypeScript, DTO validés, accès à la base via ORM, stores dédiés et utilisation de composants réutilisables. Le projet prévoit également une distinction claire entre le contenu permanent du système et les informations susceptibles d'être modifiées par l'administration.

La qualité du code dépend toutefois autant de la discipline de développement que des outils choisis. Des conventions de nommage, une gestion régulière des erreurs, une journalisation adaptée et une stratégie de tests doivent accompagner l'architecture pour éviter que la complexité n'augmente avec les fonctionnalités.

IX Conclusion du chapitre

Le socle technologique retenu répond aux différentes exigences du projet. Le back-end est adapté à une architecture modulaire et à la communication temps réel, tandis que le front-end permet de construire une interface réactive et graphiquement avancée. La persistance, les médias et la messagerie complètent cet ensemble. Le chapitre suivant peut ainsi se concentrer sur la partie la plus sensible du système : l'intelligence artificielle vocale temps réel.

Chapitre 5 :

Intelligence artificielle temps réel et expérience immersive

I Introduction du chapitre

La composante conversationnelle constitue l'élément technique le plus spécifique du projet. Elle combine plusieurs flux simultanés : audio entrant, messages de contrôle, appels d'outils, données de connaissances, audio sortant et événements d'état. La réussite de cette partie dépend autant du modèle utilisé que de l'organisation logicielle qui l'entoure.

II Principe du pipeline vocal

Le cycle commence dans le navigateur. Le micro est obtenu via les API audio du navigateur, puis les échantillons sont convertis en PCM. Les paquets sont transmis au serveur par WebSocket. Le serveur dispose d'une passerelle qui joue le rôle de relais entre le client et le service conversationnel distant.

Lorsque le modèle produit une réponse audio, le flux est renvoyé vers le navigateur. Le front-end ordonne les buffers reçus afin d'éviter les interruptions sonores. En cas d'interruption de la conversation, les buffers non joués peuvent être arrêtés pour donner une sensation naturelle de reprise de parole.

Table 5.1 Principales caractéristiques du flux audio

Paramètre	Valeur / principe décrit dans les spécifications
Entrée	Microphone du navigateur
Format entrant	PCM linéaire, échantillonnage 16 kHz
Transport	WebSocket bidirectionnel
Encodage transport	Base64 pour les paquets audio
Sortie	Flux audio PCM, échantillonnage 24 kHz
Restitution	Web Audio API
Interruption	Arrêt immédiat des buffers en cours
Protection acoustique	Blocage logiciel du micro pendant la parole de l'agent

III Passerelle WebSocket

La passerelle constitue le point central du dialogue. À l'ouverture d'une session, elle établit une connexion avec le service conversationnel et envoie la configuration initiale : modèle, voix, instructions et outils disponibles. Elle reçoit ensuite les paquets de l'utilisateur, les relaie et interprète les messages de retour.

La passerelle ne se limite donc pas à transporter des bytes. Elle doit également gérer le protocole de session, les appels d'outils, les événements d'interruption et le transfert des états émotionnels. Elle représente à ce titre une couche d'orchestration entre l'interface et le service d'intelligence artificielle.

IV Tool Calling

Le projet adopte une stratégie de récupération à la demande. Au lieu d'injecter l'ensemble des informations métier dans les instructions système, le modèle dispose d'un ensemble de fonctions. Lorsqu'un utilisateur demande une information, le modèle décide s'il doit invoquer une fonction avant de formuler une réponse.

Cette approche présente plusieurs bénéfices. Les informations restent centralisées dans la base de données, les réponses peuvent refléter les mises à jour récentes, et le contexte initial du modèle reste plus compact. Le système sépare ainsi la compétence conversationnelle du stockage de la connaissance métier.

Table 5.2 Fonctions du mécanisme de Tool Calling

Fonction	Données récupérées	Usage
`getCompanyInfo`	Profil, mission, vision	Questions générales sur l'organisation
`getServices`	Services et caractéristiques	Présentation d'une offre
`getPricing`	Tarifs et délais	Demandes commerciales
`getProjects`	Portfolio et études de cas	Questions sur les réalisations
`getFaq`	Questions/réponses	Interrogations fréquentes
`setEmotion`	État émotionnel	Synchronisation visuelle

V Cache mémoire

Les informations sollicitées pendant la conversation peuvent être préchargées en mémoire au démarrage du serveur. L'objectif est de réduire les accès répétitifs à PostgreSQL pendant les tours de parole. Le cache représente une copie rapide des données de consultation et peut être rafraîchi lorsque l'administration modifie les informations métier.

Cette stratégie est particulièrement adaptée aux contenus relativement stables tels que les services, les tarifs et les questions fréquentes. Elle doit cependant être accompagnée d'une politique d'invalidation claire afin d'éviter qu'une valeur ancienne ne reste visible après une mise à jour.

VI États émotionnels

Le système prévoit six états émotionnels principaux : neutral, happy, confident, thinking, friendly et serious. Ces états ne constituent pas un diagnostic psychologique ; ils représentent simplement des modes d'expression visuelle utilisés pour synchroniser l'avatar avec le dialogue.

Table 5.3 Etats visuels associés aux interactions

Etat	Utilisation visuelle ou conversationnelle
neutral	Situation par défaut ou transition
happy	Réponse positive, accueil, remerciement
confident	Information présentée avec assurance
thinking	Recherche d'une information ou traitement d'un appel d'outil
friendly	Interaction chaleureuse et accessible
serious	Sujet technique ou information nécessitant davantage de précision



Figure 5.1 États émotionnels de l'agent conversationnel

VII Protection contre l'écho et interruption

Une conversation vocale dans un navigateur doit éviter que la voix de l'agent soit réinjectée dans le microphone de l'utilisateur. Le hook audio mesure l'activité de lecture et désactive temporairement la capture pendant la parole de l'agent ou juste après son arrêt. Cette stratégie réduit le risque d'une boucle acoustique artificielle.

Le mécanisme de barge-in, c'est-à-dire l'interruption par l'utilisateur, constitue l'autre point essentiel. Lorsqu'une interruption est détectée, le serveur transmet l'événement au front-end qui arrête les sources audio actuellement jouées. La réponse non terminée disparaît alors immédiatement, permettant au dialogue de repartir sur la nouvelle prise de parole.

VIII Séquence d'exécution

Le cycle de traitement peut être décrit textuellement en six étapes : capture, encodage, relais, inférence, exécution éventuelle des outils, puis génération et restitution audio. À chacune de ces étapes, la latence et la robustesse du protocole doivent être surveillées.

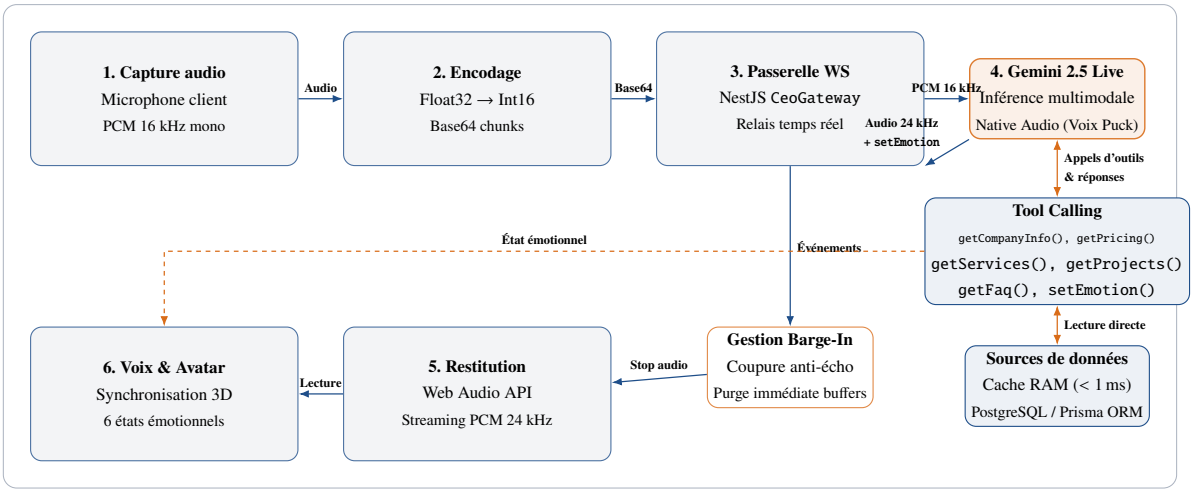


Figure 5.2 Cycle d'exécution du pipeline audio temps réel et boucle d'orchestration

IX Expérience immersive et rendu 3D

La dimension immersive s'appuie sur un ensemble de technologies graphiques. Three.js est utilisé pour la scène 3D, avec des shaders GLSL personnalisés. React Three Fiber permet de déclarer la scène sous forme de composants React. Un globe interactif peut être enrichi par un éclairage atmosphérique, tandis qu'un carrousel cylindrique est rendu avec OGL.

Les animations de navigation utilisent GSAP et ScrollTrigger. Le défilement fluide fourni par Lenis est synchronisé avec le système d'animation. L'objectif n'est pas de multiplier les effets, mais de construire un mouvement cohérent entre les différentes sections et de mettre en valeur les contenus.

X Gestion des pages et de l'expérience

La page d'accueil assemble plusieurs sections : préchargement, présentation principale, expertises, portfolio, méthode, prise de contact, FAQ et localisation. Le portfolio dispose d'une page générale et de pages détaillées. Le blog possède également une vue de liste et une vue détaillée. Les solutions métier sont présentées dans une zone dédiée et la page de contact reprend le même langage visuel que le reste de l'expérience.

XI Administration de la base de connaissances

L'administration joue un rôle central dans l'actualisation de l'agent. L'opérateur peut modifier un service, un tarif ou une FAQ, enregistrer la modification et provoquer la mise à jour du cache. Le modèle n'a donc pas besoin d'être réentraîné pour chaque changement de contenu métier.

Table 5.4 Domaines administrables dans la base de connaissances

Domaine	Informations gérées
Profil	Nom, coordonnées, présentation, philosophie, mission, vision
Services	Catégories, titres, descriptions, fonctionnalités
Tarifs	Offres, prix de départ, délais, livrables
FAQ	Catégories, questions, réponses, ordre d'affichage
Portfolio	Projets, contexte, solution, résultats, médias

XII Conclusion du chapitre

La composante IA démontre que l'expérience temps réel repose sur une chaîne complète et non sur le seul modèle de langage. La capture audio, le transport, la passerelle, les outils, le cache, la génération sonore et l'affichage visuel fonctionnent comme un ensemble. La section suivante évalue la réalisation obtenue et présente les méthodes de validation ainsi que les limites restantes.

Chapitre 6 :

Réalisation, validation, résultats et perspectives

I Introduction du chapitre

La dernière étape du projet consiste à confronter la conception aux fonctionnalités effectivement implémentées. L'analyse porte sur les principaux modules, le comportement des parcours utilisateurs, l'administration, la communication temps réel et la qualité de l'expérience. Cette étape permet également de distinguer ce qui est opérationnel de ce qui relève encore d'une amélioration future.

II Fonctionnalités réalisées

La réalisation couvre plusieurs domaines. La partie publique présente les contenus, les services, les réalisations et les articles. La partie administrative permet de gérer les informations sans modifier le code applicatif. Le formulaire de contact alimente la persistance et déclenche une notification. Enfin, le module conversationnel fournit une interaction vocale et peut consulter la connaissance métier au moyen d'outils dédiés.

Table 6.1 Bilan fonctionnel de la réalisation

Fonctionnalité	État	Observations
Page d'accueil	Réalisée	Assemblage des principales sections
Portfolio	Réalisée	Liste, filtrage et détail des projets
Blog	Réalisée	Publication, catégories et consultation
Contact	Réalisée	Enregistrement et notification
Administration des contenus	Réalisée	Gestion des projets et articles
Base de connaissances	Réalisée	Profil, services, tarifs et FAQ
Authentification administrative	Réalisée	OTP temporaire par courrier
Conversation vocale	Réalisée	Flux audio bidirectionnel
Appels d'outils	Réalisée	Récupération ciblée des informations
Etats émotionnels	Réalisée	Synchronisation de l'état côté client
Expérience 3D	Réalisée	Scène et animations graphiques
Optimisation production	En évolution	Plusieurs améliorations restent possibles

III Validation fonctionnelle

La validation peut être organisée autour de scénarios représentatifs. Pour la partie publique, il s'agit de vérifier la navigation, l'ouverture des détails, la consultation du blog et l'envoi du formulaire. Pour l'administration, il faut vérifier l'authentification, la création, la modification et la suppression des contenus ainsi que la mise à jour de la connaissance.

Pour la conversation, les scénarios critiques sont différents : démarrage de session, capture audio, transmission, appel d'outil, réception audio, changement d'état émotionnel, interruption et reprise. Chaque scénario doit être vérifié indépendamment puis dans une séquence complète.

Table 6.2 Plan de tests fonctionnels

ID	Cas de test	Résultat attendu	Priorité
T01	Ouvrir la page principale	Les sections principales sont accessibles	Haute
T02	Consulter un projet	Le détail est chargé correctement	Haute
T03	Lire un article	Le contenu et les métadonnées sont visibles	Moyenne
T04	Envoyer un contact	La demande est enregistrée et notifiée	Haute
T05	Accéder à l'administration	Le mécanisme OTP contrôle l'accès	Haute
T06	Modifier un service	La nouvelle valeur est persistée	Haute
T07	Démarrer une session vocale	Un canal WebSocket est établi	Haute
T08	Demander une information tarifaire	L'outil de tarification est sollicité	Haute
T09	Interrompre l'agent	L'audio courant est arrêté	Haute
T10	Changer une information métier	Le cache est rafraîchi	Haute

IV Validation des données

La persistance est validée par des vérifications de cohérence sur les entités principales. Les identifiants doivent rester uniques, les champs obligatoires doivent être contrôlés et les dates de création ou de mise à jour doivent être cohérentes. Les données multilingues doivent conserver une structure homogène afin d'être exploitées aussi bien par l'interface que par les outils conversationnels.

V Validation du flux audio

Le flux audio est validé par plusieurs critères : capacité à ouvrir la session, continuité de transmission, absence de coupures importantes, traitement des interruptions et correspondance entre l'état de la conversation et l'état visuel. La validation doit être effectuée dans des conditions proches de l'usage réel, car les performances d'un flux réseau peuvent varier selon le navigateur et l'environnement.

Table 6.3 Critères de validation du temps réel

Critère	Indicateur de vérification
Connexion	Session WebSocket correctement ouverte et fermée
Transmission	Réception régulière des paquets audio
Continuité	Absence de blancs anormaux dans la restitution
Réactivité	Réponse dans un délai compatible avec l'interaction
Tool Calling	Appel correct des fonctions pertinentes
Synchronisation	Etat visuel cohérent avec le dialogue
Barge-in	Arrêt immédiat du son lors d'une interruption
Sécurité	Accès administratif protégé

VI Résultats obtenus

Sur le plan logiciel, la séparation des modules permet d'obtenir une plateforme dans laquelle les contenus, la logique métier et l'interface évoluent de manière relativement indépendante. Le modèle de persistance permet de conserver les données structurées et les contenus multilingues, tandis que le cache accélère la lecture des informations mobilisées par les appels d'outils.

Sur le plan expérientiel, l'association du rendu graphique, des animations et de la conversation vocale transforme la plateforme en démonstrateur technologique. L'expérience ne se limite pas à présenter une liste de technologies : elle met ces technologies au service d'une interaction concrète.

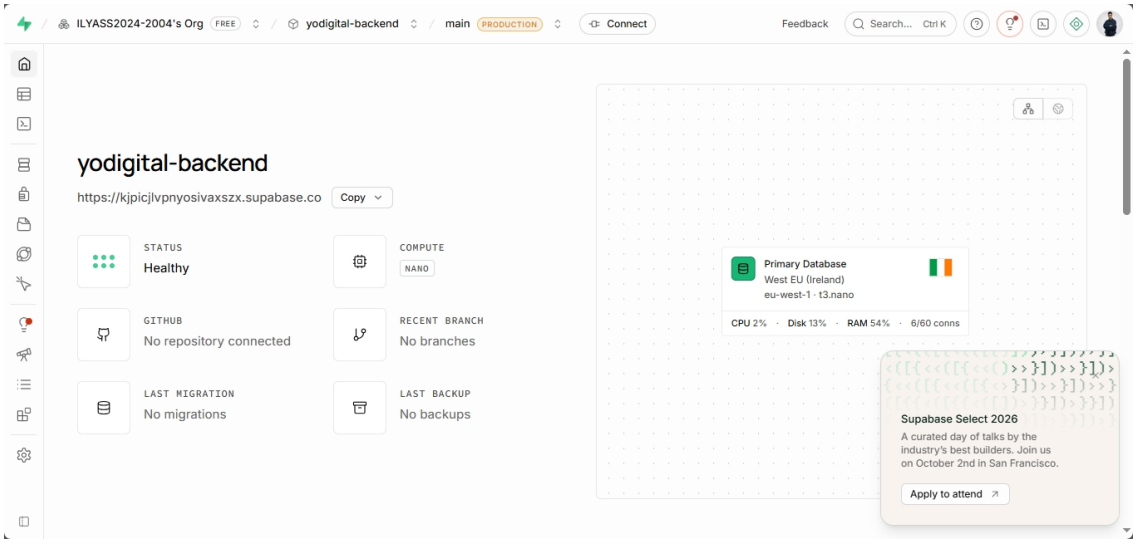


Figure 6.1 Résultat de la base de données



FIGURE 6.2 Résultat du back-end : structure des modules et services



FIGURE 6.3 Résultat du front-end : interfaces et parcours principaux

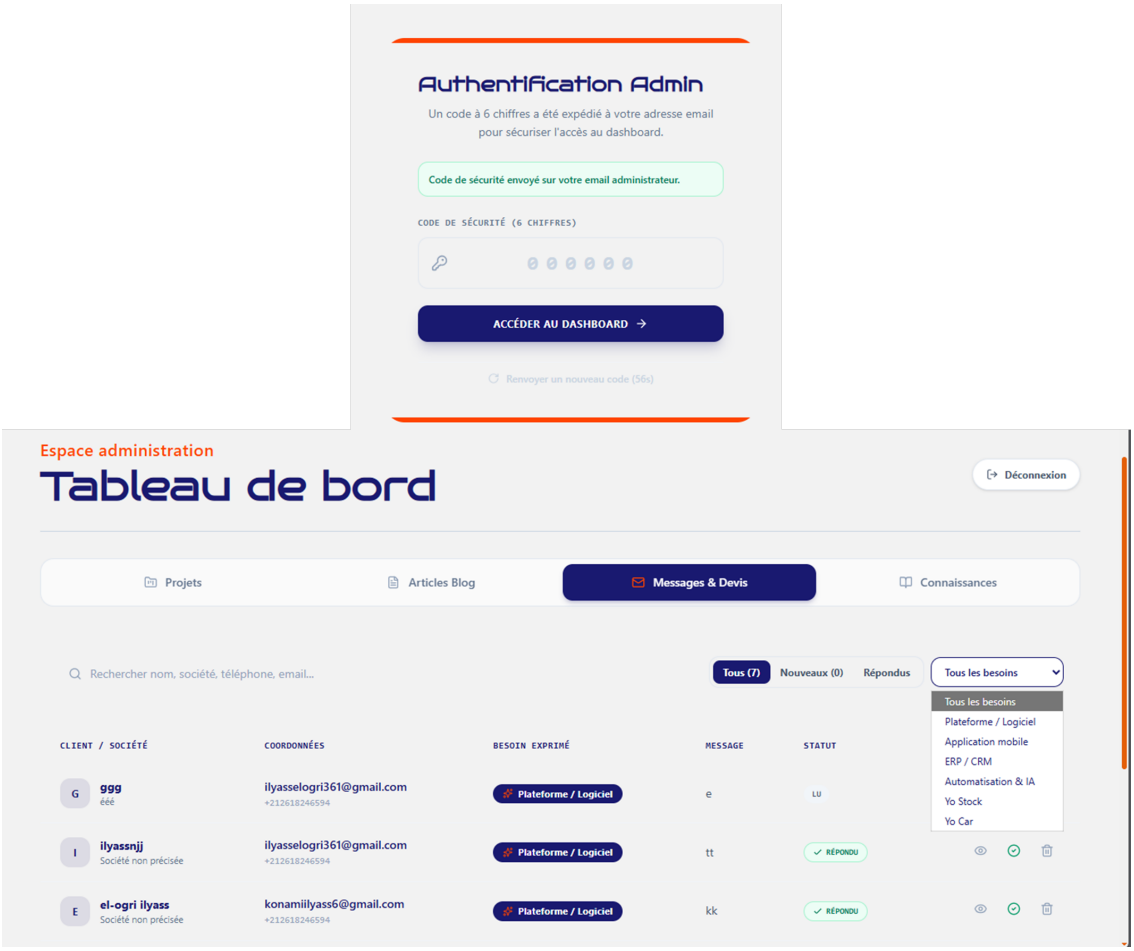


FIGURE 6.4 Résultat de l'espace d'administration

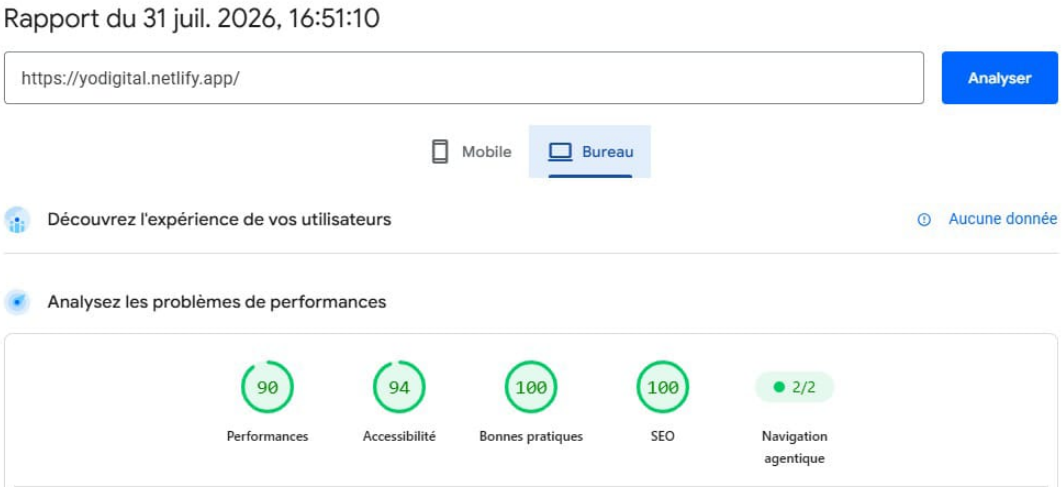


FIGURE 6.5 Synthèse des résultats

VII Limites identifiées

Plusieurs limites apparaissent naturellement lorsqu'un tel système est envisagé dans une logique de production. La première concerne le cache mémoire local. Une architecture multi-instance ne peut pas garantir une synchronisation uniforme si chaque processus conserve

sa propre copie. La deuxième concerne le traitement audio dans le navigateur : l'utilisation d'un mécanisme exécuté sur le thread principal peut devenir sensible à la charge graphique.

La troisième limite concerne l'expérience mobile. Le rendu 3D riche et le traitement audio simultané peuvent être coûteux sur des appareils moins puissants. Une stratégie adaptative est donc nécessaire pour ajuster le niveau de détail, les effets et le nombre d'animations en fonction des capacités du terminal.

VIII Pistes d'amélioration

VIII.1 Limitation des sessions de conversation IA

Afin de maîtriser les coûts de consommation de l'API vocale multimodale, l'accès visiteur doit être encadré : chaque utilisateur dispose d'une session unique plafonnée à **15 minutes**. Ce délai écoulé, la communication est automatiquement clôturée, empêchant la monopolisation des ressources ou les connexions successives abusives.

VIII.2 Identification de la session par cookie

Un cookie technique déposé lors de la première interaction enregistre l'état de consommation de la session. Lors d'une nouvelle tentative, le serveur contrôle l'existence et la validité de ce marqueur pour refuser la réouverture d'un flux audio, limitant ainsi les abus sans imposer d'authentification préalable.

VIII.3 Vérification complémentaire par adresse IP

En renfort du cookie, l'adresse IP constitue un second signal pour détecter la suppression manuelle du traceur local. Utilisée comme filtre d'appoint, cette vérification prend en compte les contraintes des réseaux partagés (NAT, proxys) pour éviter les faux positifs tout en limitant les contournements triviaux.

VIII.4 Contrôle et temporisation côté serveur

La validation des accès est appliquée côté serveur afin d'empêcher toute altération du client. Le serveur évalue conjointement l'historique de la session, l'état du jeton et les informations réseau. Parallèlement, un minuteur serveur met fin unilatéralement à la connexion WebSocket dès l'atteinte des 15 minutes ou en cas d'inactivité prolongée.

VIII.5 Réduction de la consommation et maîtrise des coûts

Le traitement vocal temps réel sollicitant simultanément le réseau bidirectionnel, le modèle génératif et le moteur d'inférence, cette politique de restriction (durée bornée, sessions uniques et fermeture des sockets orphelins) préserve la fluidité de la démonstration tout en garantissant un coût d'exploitation soutenable.

VIII.6 Observabilité et suivi de la consommation

L'instrumentation du système fournirait des métriques clés : durée moyenne des échanges, taux de refus, volumétrie des appels d'outils, fréquence du *barge-in* et erreurs réseau. Ces données permettraient d'ajuster finement les quotas selon l'usage réel constaté.

VIII.7 Tests automatisés

L'intégration de tests automatisés (unitaires et d'intégration) sécuriserait les modules critiques : passerelle WebSocket, authentification, routes d'administration et scénarios de limitation (expiration du délai, reconnexion, altération du cookie). Cette couverture fiabiliserait le système face aux régressions techniques.

Table 6.4 Synthèse des améliorations proposées

Amélioration	Problème ciblé	Bénéfice attendu
Session limitée à 15 min	Échanges vocaux excessifs	Maîtrise budgétaire des requêtes IA
Cookie de session	Réouvertures intempestives	Blocage direct sans authentification requise
Filtrage IP complémentaire	Contournement par purge du cookie	Consolidation du contrôle d'accès
Validation serveur	Altérations côté navigateur	Application stricte des règles de session
Temporisation WebSocket	Sockets inactives ou persistantes	Libération rapide des ressources serveur
Observabilité applicative	Manque de métriques d'usage	Pilotage analytique de la charge et des coûts
Tests automatisés	Risques de régression	Fiabilité accrue de la logique métier et temps réel

IX Bilan technique

Le bilan met en évidence la cohérence entre les objectifs et les choix effectués. La plateforme repose sur une architecture moderne capable de supporter à la fois des flux applicatifs classiques et une interaction temps réel. La base de données fournit une structure suffisamment flexible pour gérer les contenus métier, tandis que le mécanisme d'outils maintient une séparation entre le modèle conversationnel et la source de vérité.

La partie graphique ajoute une dimension démonstrative importante, mais elle doit rester soumise aux contraintes de performance. La qualité d'une expérience immersive ne dépend pas

seulement du nombre d'effets affichés ; elle dépend aussi de la régularité des animations, de la stabilité de l'audio et de la facilité avec laquelle l'utilisateur comprend les interactions.

X Conclusion du chapitre

La réalisation atteint les principaux objectifs définis dans le cadrage. Les composants publics, administratifs et conversationnels sont structurés autour d'une architecture cohérente. Les validations fonctionnelles permettent d'identifier les scénarios critiques et les limites restantes. Les pistes proposées fournissent une trajectoire claire pour une évolution vers une solution plus robuste, observable et adaptée à un déploiement à plus grande échelle.

Conclusion générale

Ce Projet de Fin d'Année a permis d'étudier et de mettre en œuvre une plateforme numérique réunissant plusieurs domaines complémentaires de l'ingénierie logicielle. L'approche n'a pas consisté à juxtaposer des technologies, mais à les articuler autour d'une expérience utilisateur et d'une architecture cohérente.

La première dimension du travail a concerné l'analyse du besoin et la définition du périmètre. Cette étape a permis de distinguer les fonctionnalités publiques, les fonctions administratives et la composante conversationnelle. La deuxième dimension a porté sur la conception logicielle, avec une architecture séparant clairement le front-end, le back-end, la persistance et les services externes.

La troisième dimension a concerné la réalisation des modules applicatifs : gestion du portfolio, blog, contacts, administration et base de connaissances. La quatrième dimension a été consacrée au temps réel et à l'intelligence artificielle. Le pipeline audio, le WebSocket, le Tool Calling, le cache mémoire et la synchronisation des états montrent que l'IA conversationnelle doit être considérée comme un sous-système complet.

Enfin, la dimension immersive a apporté une réflexion sur la place du rendu 3D et des animations dans une application moderne. Les technologies graphiques offrent de nouvelles possibilités de présentation, mais leur utilisation doit rester équilibrée avec les exigences de performance et d'accessibilité.

Au terme du projet, l'architecture obtenue constitue une base évolutive. Les travaux futurs pourront renforcer la robustesse opérationnelle, l'observabilité, le support mobile, les tests automatisés et la distribution du cache. La démarche acquise au cours de ce projet représente également un apport méthodologique important : elle permet d'aborder les futurs systèmes en considérant simultanément les besoins, les données, les services, l'expérience et les contraintes d'exploitation.

Webographie

Technologie / Domaine	Référence utilisée dans le projet
React	https://react.dev/
React – Documentation d'apprentissage	https://react.dev/learn/
React – API Reference	https://react.dev/reference/react
React Router	https://reactrouter.com/
React Router – Documentation	https://reactrouter.com/home
Tailwind CSS	https://tailwindcss.com/
Tailwind CSS – Installation	https://tailwindcss.com/docs/installation
Tailwind CSS – Vite	https://tailwindcss.com/docs/installation/using-vite
Vite	https://vite.dev/
Vite – Guide officiel	https://vite.dev/guide/
TypeScript	https://www.typescriptlang.org/
TypeScript – Handbook	https://www.typescriptlang.org/docs/handbook/intro.html
Axios	https://axios-http.com/
Axios – Documentation	https://axios-http.com/docs/intro
Zustand	https://zustand.docs.pmnd.rs/
Zustand – Documentation	https://zustand.docs.pmnd.rs/learn/
Lucide	https://lucide.dev/
Lucide React	https://lucide.dev/guide/packages/lucide-react
Three.js	https://threejs.org/

Suite à la page suivante

Technologie / Domaine	Référence utilisée dans le projet
Three.js – Documentation	https://threejs.org/docs/
Three.js – Manual	https://threejs.org/manual/
React Three Fiber	https://r3f.docs.pmnd.rs/
React Three Fiber – Documentation	https://docs.pmnd.rs/react-three-fiber
Drei	https://drei.docs.pmnd.rs/
React Three Postprocessing	https://react-postprocessing.docs.pmnd.rs/
GSAP	https://gsap.com/
GSAP – Documentation	https://gsap.com/docs/v3/
GSAP ScrollTrigger	https://gsap.com/docs/v3/Plugins/ScrollTrigger/
Lenis	https://github.com/darkroomengineering/lenis
OGL	https://github.com/oframe/ogl
Lottie Web	https://github.com/airbnb/lottie-web
LottieFiles	https://lottiefiles.com/
Vite Plugin GLSL	https://github.com/UstymUkhman/vite-plugin-glsl
Node.js	https://nodejs.org/
Node.js – Documentation	https://nodejs.org/docs/latest/api/
NestJS	https://nestjs.com/
NestJS – Documentation	https://docs.nestjs.com/
NestJS – Modules	https://docs.nestjs.com/modules
NestJS – WebSockets	https://docs.nestjs.com/websockets/gateways
NestJS – Validation	https://docs.nestjs.com/techniques/validation
NestJS – Cookies	https://docs.nestjs.com/techniques/cookies
Express	https://expressjs.com/
WebSocket	https://developer.mozilla.org/en-US/docs/Web/API/WebSocket

Suite à la page suivante

Technologie / Domaine	Référence utilisée dans le projet
WebSocket API – MDN	https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
WebSocket – RFC 6455	https://www.rfc-editor.org/rfc/rfc6455
Axios HTTP Client	https://axios-http.com/docs/intro
Prisma	https://www.prisma.io/
Prisma – Documentation	https://www.prisma.io/docs
Prisma – Getting Started	https://www.prisma.io/docs/getting-started
Prisma – ORM Guides	https://www.prisma.io/docs/guides
Prisma – Schema Reference	https://www.prisma.io/docs/orm/reference/prisma-schema-reference
PostgreSQL	https://www.postgresql.org/
PostgreSQL – Documentation	https://www.postgresql.org/docs/
PostgreSQL – JSON	https://www.postgresql.org/docs/current/datatype-json.html
Supabase	https://supabase.com/
Supabase – Documentation	https://supabase.com/docs
OWASP	https://owasp.org/
OWASP Authentication Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
OWASP Session Management Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
OWASP API Security Top 10	https://owasp.org/API-Security/
OWASP API Security Top 10 – 2023	https://api-security.owasp.org/editions/2023/en/0x11-t10/
JSON Web Token (JWT)	https://www.jwt.io/

Suite à la page suivante

Technologie / Domaine	Référence utilisée dans le projet
JWT – Introduction	https://www.jwt.io/introduction
Bcrypt	https://www.npmjs.com/package/bcrypt
MDN – HTTP Cookies	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies
MDN – Cookie Header	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cookie
MDN – CORS	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS
MDN – Fetch API	https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
MDN – HTTP	https://developer.mozilla.org/en-US/docs/Web/HTTP
MDN – HTTPS	https://developer.mozilla.org/en-US/docs/Glossary/HTTPS
Web Audio API	https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
Web Audio API – Concepts	https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Basic_concepts_behind_Web_Audio_API
Web Audio API – Guide	https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Using_Web_Audio_API
MediaDevices	https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices
getUserMedia	https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
Web Speech API	https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
WebGL	https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API
WebGL – Tutoriel	https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/Tutorial
Khronos Group	https://www.khronos.org/

Suite à la page suivante

Technologie / Domaine	Référence utilisée dans le projet
WebGL Specification	https://registry.khronos.org/webgl/specs/latest/
OpenGL ES	https://www.khronos.org/opengles/
GLSL Specification	https://registry.khronos.org/OpenGL/specs/gl/
Google AI for Developers	https://ai.google.dev/
Gemini API	https://ai.google.dev/gemini-api/docs
Gemini Live API	https://ai.google.dev/gemini-api/docs/live
Gemini Function Calling	https://ai.google.dev/gemini-api/docs/function-calling
Gemini API – Audio	https://ai.google.dev/gemini-api/docs/audio
Google AI Studio	https://aistudio.google.com/
Cloudinary	https://cloudinary.com/
Cloudinary – Documentation	https://cloudinary.com/documentation
Cloudinary – Node.js	https://cloudinary.com/documentation/node_integration
Cloudinary – Upload API	https://cloudinary.com/documentation/image_upload_api_reference
Nodemailer	https://nodemailer.com/
Nodemailer – Usage	https://nodemailer.com/usage
Stripe	https://stripe.com/
Stripe – Documentation	https://docs.stripe.com/
Stripe – API Reference	https://docs.stripe.com/api
Docker	https://www.docker.com/
Docker – Documentation	https://docs.docker.com/
Docker – Get Started	https://docs.docker.com/get-started/
PlantUML	https://plantuml.com/
PlantUML – Guide	https://plantuml.com/guide

Suite à la page suivante

Technologie / Domaine	Référence utilisée dans le projet
PlantText	https://www.planttext.com/
Git	https://git-scm.com/
Git – Documentation	https://git-scm.com/docs
GitHub	https://github.com/
Google Search Central	https://developers.google.com/search
Google Search Central – SEO Starter Guide	https://developers.google.com/search/docs/fundamentals/seo-starter-guide
Google Search Central – Structured Data	https://developers.google.com/search/docs/appearance/structured-data/intro-structured-data
Schema.org	https://schema.org/
Open Graph Protocol	https://ogp.me/
Google Analytics	https://analytics.google.com/
ECMAScript Specification	https://tc39.es/ecma262/
MDN Web APIs	https://developer.mozilla.org/en-US/docs/Web/API
MDN JavaScript Guide	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
W3C	https://www.w3.org/
WHATWG	https://whatwg.org/
RFC Editor	https://www.rfc-editor.org/